

 2. Basic Concepts.....	2
 3. Important System Calls.....	2
3.1. fork().....	2
3.2. getpid() and getppid().....	3
3.3. wait() and waitpid().....	3
3.4. exec Family of Functions.....	4
 4. Advanced Concepts.....	4
4.1. Creating a Process Tree.....	5
4.2. Zombie Processes.....	5
4.3. Orphan Processes.....	6
4.4. Signals and kill().....	6
 5. Memory Allocation Basics.....	7
 6. Complete Practice Problems.....	7
 Quick Commands + Cheat Sheet (Summary).....	9
Bash Terminal Commands.....	9
System Programming Functions.....	9
🚀 Now you have a huge, complete, exam-grade open-book resource — focused only on what you need.....	10
Shell Scripting Questions.....	11
System Programming (Linux C Programming) Questions.....	13
Shell Scripting Questions.....	18
System Programming (Linux C Programming) Questions.....	20
Shell Scripting Questions:.....	24
System Programming Questions:.....	24
System Programming Solutions.....	26
1. Print PID and PPID:.....	26
2. Create child processes and parent waits:.....	26
3. Memory allocation using malloc, calloc, and free:.....	27
4. Create 5 child processes:.....	28
5. Replace current process using exec():.....	29
6. Using wait() to make parent wait for child:.....	29
7. Child process using exit():.....	30
8. Memory leak example:.....	30
9. Fork and print user input in child process:.....	31
10. Process creation and parent waits:.....	32
System Programming Questions.....	33
Process Management (Chapter 6).....	33
Memory Allocation (Chapter 7).....	33
Process Creation (Chapter 24).....	33
Process Termination (Chapter 25).....	34
Monitoring Child Processes (Chapter 26).....	34
General System Programming.....	34
Comprehensive Practical Problems.....	35
System Programming Solutions.....	36
Process Management (Chapter 6).....	36

Memory Allocation (Chapter 7).....38

Process Creation and Termination (Chapters 24-26)..... 38

2. Basic Concepts

Term	Meaning
Process	Running instance of a program
Parent Process	The original process
Child Process	Process created by parent
PID	Process ID (unique number for each process)
PPID	Parent Process ID
Fork	System call to create a child process
Exec	Replace current process image with new program
Wait	Make parent wait for child to finish
Zombie Process	Dead process but entry in process table
Orphan Process	Parent terminated, but child still running

3. Important System Calls

3.1. fork()

- Creates a **new process** (child) by duplicating the calling process.
- Syntax:

```
pid_t fork(void);
```

- Returns:
 - > 0 → Parent process (returns child's PID)
 - $= 0$ → Child process
 - < 0 → Fork failed

 Example:

```
#include <stdio.h>
#include <unistd.h>

int main() {
    pid_t pid = fork();
    if(pid < 0) {
        printf("Fork failed!\n");
    }
    else if(pid == 0) {
        printf("Child Process! PID: %d, PPID: %d\n", getpid(), getppid());
    }
    else {
        printf("Parent Process! PID: %d\n", getpid());
    }
    return 0;
}
```

3.2. getpid() and getppid()

- **getpid()** → Get the **PID** of the current process.
- **getppid()** → Get the **PID** of the parent process.

✓ Example inside any program:

```
printf("My PID: %d\n", getpid());
printf("Parent PID: %d\n", getppid());
```

3.3. wait() and waitpid()

- Make parent **wait** until the child terminates.
- Syntax:

```
pid_t wait(int *status);
pid_t waitpid(pid_t pid, int *status, int options);
```

✓ Example:

```
#include <stdio.h>
#include <sys/wait.h>
#include <unistd.h>
```

```
int main() {
    pid_t pid = fork();
```

```

if(pid == 0) {
    printf("Child Process\n");
    sleep(2);
} else {
    wait(NULL); // Parent waits
    printf("Parent Process after child exits\n");
}
return 0;
}

```

3.4. exec Family of Functions

Replace current process image with a new program.

Function	Description
execl()	Execute program
execp()	Search for program in PATH
execvp()	Vector arguments
execv()	Vector without PATH search

Syntax (example):

```

int execl(const char *path, const char *arg0, ..., (char *)0);
int execvp(const char *file, char *const argv[]);

```

✓ Example:

```

#include <stdio.h>
#include <unistd.h>

int main() {
    printf("Before exec\n");
    execlp("ls", "ls", "-l", NULL);
    printf("This line will not print if exec is successful\n");
    return 0;
}

```



4. Advanced Concepts

4.1. Creating a Process Tree

✓ Example:

Parent → Child → Grandchild

```
#include <stdio.h>
#include <unistd.h>

int main() {
    pid_t pid = fork();
    if(pid == 0) {
        printf("Child created. PID: %d\n", getpid());
        pid_t pid2 = fork();
        if(pid2 == 0) {
            printf("Grandchild created. PID: %d\n", getpid());
        }
    } else {
        wait(NULL);
    }
    return 0;
}
```

4.2. Zombie Processes

- When child terminates, but parent **does not call wait()** → zombie.

✓ Example:

```
#include <stdio.h>
#include <unistd.h>

int main() {
    pid_t pid = fork();
    if(pid == 0) {
        printf("Child exiting...\n");
    } else {
        sleep(10); // Parent is sleeping
        printf("Parent running. Check with ps command.\n");
    }
    return 0;
}
```

Run `ps aux | grep a.out` to see `<defunct>` zombie.

4.3. Orphan Processes

- When parent terminates, but child still running.

✓ Example:

```
#include <stdio.h>
#include <unistd.h>

int main() {
    pid_t pid = fork();
    if(pid > 0) {
        printf("Parent exiting...\n");
    } else {
        sleep(5);
        printf("Child (Orphan) now. New PPID: %d\n", getppid());
    }
    return 0;
}
```

When child prints, its **PPID will be 1** (init process).

4.4. Signals and kill()

- `kill(pid, signal)` → Send signal to a process.

✓ Example:

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

int main() {
    printf("PID: %d\n", getpid());
    while(1);
}
```

- Open another terminal and run:

`kill -9 PID`

to forcibly kill.

5. Memory Allocation Basics

- **malloc()** - allocate memory at runtime.
- **free()** - release memory.

✓ Example:

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *p;
    p = (int*)malloc(5 * sizeof(int));
    if(p == NULL) {
        printf("Memory allocation failed\n");
        return 1;
    }
    for(int i = 0; i < 5; i++) {
        p[i] = i+1;
        printf("%d ", p[i]);
    }
    free(p);
    return 0;
}
```

6. Complete Practice Problems

✓ **Problem 1:** Write a C program that creates 5 child processes from the same parent.

```
#include <stdio.h>
#include <unistd.h>

int main() {
    for(int i = 0; i < 5; i++) {
        pid_t pid = fork();
        if(pid == 0) {
            printf("Child %d, PID: %d\n", i+1, getpid());
            return 0;
        }
    }
    for(int i = 0; i < 5; i++) {
        wait(NULL);
    }
    return 0;
}
```

✓ **Problem 2:** Create a C program where parent waits for child and child replaces itself with `date` command.

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();
    if(pid == 0) {
        execlp("date", "date", NULL);
    } else {
        wait(NULL);
        printf("Child finished executing date\n");
    }
    return 0;
}
```

✓ **Problem 3:** Demonstrate a zombie process.

```
#include <stdio.h>
#include <unistd.h>

int main() {
    pid_t pid = fork();
    if(pid == 0) {
        printf("Child exiting...\n");
    } else {
        sleep(10);
        printf("Parent alive, child zombie\n");
    }
    return 0;
}
```

✓ **Problem 4:** Demonstrate an orphan process.

```
#include <stdio.h>
#include <unistd.h>

int main() {
    pid_t pid = fork();
    if(pid > 0) {
        printf("Parent exiting...\n");
    } else {
        sleep(5);
        printf("Child running with new PPID %d\n", getppid());
    }
    return 0;
}
```

```
}
```

✓ **Problem 5:** Allocate memory dynamically using malloc.

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *arr = malloc(10 * sizeof(int));
    if(arr == NULL) {
        printf("Memory allocation failed\n");
        return 1;
    }
    for(int i = 0; i < 10; i++) {
        arr[i] = i * 2;
        printf("%d ", arr[i]);
    }
    free(arr);
    return 0;
}
```



Quick Commands + Cheat Sheet (Summary)

Bash Terminal Commands

Command	Meaning
<code>pwd</code>	Print working directory
<code>ls -l</code>	List files with details
<code>mkdir dir</code>	Make new directory
<code>rm file</code>	Remove file
<code>chmod +x script.sh</code>	Make script executable
<code>./script.sh</code>	Run a bash script

System Programming Functions

Function	Purpose
----------	---------

<code>fork()</code>	Create child process
<code>getpid()</code>	Get process ID
<code>getppid()</code>	Get parent ID
<code>wait()</code>	Wait for child
<code>execlp()</code>	Replace process image
<code>malloc()</code>	Allocate memory
<code>free()</code>	Free memory
<code>kill(pid, signal)</code>	Send signal to process

 **Now you have a huge, complete, exam-grade open-book resource — focused only on what you need.**

If you want, I can also:

- **Format everything into a single printable PDF** for you (clean, easy to print).
- **Add more optional 10–20 extra practice problems** (fully solved).
- **Make a 1-page quick revision guide** as a bonus!

Sure! Here are 30-40 fully solved practice problems based on the Shell Scripting and System Programming (Linux C Programming) content you provided:

Shell Scripting Questions

1. Write a Bash script to check if a given file exists. If it does, display the message "File exists", otherwise "File not found".

```
#!/bin/bash
read -p "Enter filename: " filename
if [ -f "$filename" ]; then
    echo "File exists"
else
    echo "File not found"
fi
```

2. Create a Bash script to find the factorial of a number.

```
#!/bin/bash
read -p "Enter a number: " num
fact=1
for (( i=1; i<=num; i++ ))
do
    fact=$((fact * i))
done
echo "Factorial of $num is $fact"
```

3. Write a script to count the number of lines in a given file.

```
#!/bin/bash
read -p "Enter filename: " filename
if [ -f "$filename" ]; then
    line_count=$(wc -l < "$filename")
    echo "Number of lines in $filename is: $line_count"
else
    echo "File not found"
fi
```

4. Write a Bash script that accepts a number from the user and checks if it's even or odd.

```
#!/bin/bash
read -p "Enter a number: " num
if (( num % 2 == 0 )); then
    echo "$num is even"
else
    echo "$num is odd"
fi
```

5. Create a script to create a backup of a directory.

```
#!/bin/bash
```

```
read -p "Enter directory to backup: " dir
tar -cvf backup.tar $dir
echo "Backup of $dir created successfully"
```

6. Write a Bash script to list all the files in the current directory along with their sizes.

```
#!/bin/bash
for file in *; do
    if [ -f "$file" ]; then
        size=$(stat -c %s "$file")
        echo "$file - $size bytes"
    fi
done
```

7. Create a script to display the current system's uptime.

```
#!/bin/bash
uptime
```

8. Write a script to replace all occurrences of the word "apple" with "orange" in a given file.

```
#!/bin/bash
read -p "Enter filename: " filename
if [ -f "$filename" ]; then
    sed -i 's/apple/orange/g' "$filename"
    echo "Replaced 'apple' with 'orange' in $filename"
else
    echo "File not found"
fi
```

9. Create a script to display the user who is currently logged in.

```
#!/bin/bash
whoami
```

10. Write a script to count the number of arguments passed to it.

```
#!/bin/bash
echo "Number of arguments passed: $#"
```

11. Write a script to find the largest number among three numbers entered by the user.

```
#!/bin/bash
read -p "Enter three numbers: " num1 num2 num3
if [ $num1 -gt $num2 ] && [ $num1 -gt $num3 ]; then
    echo "$num1 is the largest"
elif [ $num2 -gt $num1 ] && [ $num2 -gt $num3 ]; then
    echo "$num2 is the largest"
```

```

else
    echo "$num3 is the largest"
fi

```

12. Create a script to display the current date in the format "Day, Month Date, Year".

```

#!/bin/bash
date +"%A, %B %d, %Y"

```

13. Write a script to search for a word in a file and display the line number(s) where the word appears.

```

#!/bin/bash
read -p "Enter filename: " filename
read -p "Enter word to search: " word
grep -n "$word" "$filename"

```

14. Create a script to check if a given number is a prime number.

```

#!/bin/bash
read -p "Enter a number: " num
flag=0
for ((i=2; i<=num/2; i++))
do
    if ((num % i == 0)); then
        flag=1
        break
    fi
done
if ((flag == 0)); then
    echo "$num is prime"
else
    echo "$num is not prime"
fi

```

15. Write a script to list the top 5 largest files in the current directory.

```

#!/bin/bash
ls -lS | head -n 5

```

System Programming (Linux C Programming) Questions

16. Write a C program that creates a child process using `fork()` and prints the process IDs.

```

#include <stdio.h>
#include <unistd.h>

int main() {
    pid_t pid = fork();

```

```

if (pid < 0) {
    printf("Fork failed!\n");
} else if (pid == 0) {
    printf("Child Process: PID = %d, PPID = %d\n", getpid(), getppid());
} else {
    printf("Parent Process: PID = %d\n", getpid());
}
return 0;
}

```

17. Write a C program that uses `exec()` to execute a new program (e.g., `ls`).

```

#include <stdio.h>
#include <unistd.h>

int main() {
    printf("Before exec\n");
    execlp("ls", "ls", "-l", NULL);
    printf("This won't print if exec is successful\n");
    return 0;
}

```

18. Write a C program that creates a child process and the parent waits for the child to finish using `wait()`.

```

#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();
    if (pid == 0) {
        printf("Child process\n");
    } else {
        wait(NULL); // Parent waits for child to finish
        printf("Parent process after child\n");
    }
    return 0;
}

```

19. Demonstrate a zombie process using `fork()`.

```

#include <stdio.h>
#include <unistd.h>

int main() {
    pid_t pid = fork();
    if (pid == 0) {
        printf("Child exiting...\n");
    } else {
        sleep(10); // Parent is sleeping
        printf("Parent alive, child zombie\n");
    }
}

```

```

    }
    return 0;
}

```

20. Write a C program that creates an orphan process.

```

#include <stdio.h>
#include <unistd.h>

int main() {
    pid_t pid = fork();
    if (pid > 0) {
        printf("Parent exiting...\n");
    } else {
        sleep(5);
        printf("Child (Orphan) now. New PPID: %d\n", getppid());
    }
    return 0;
}

```

21. Write a program to allocate memory dynamically using `malloc()` and then free it using `free()`.

```

#include <stdio.h>
#include <stdlib.h>

int main() {
    int *arr = (int *)malloc(5 * sizeof(int));
    if (arr == NULL) {
        printf("Memory allocation failed\n");
        return 1;
    }
    for (int i = 0; i < 5; i++) {
        arr[i] = i * 10;
        printf("%d ", arr[i]);
    }
    free(arr);
    return 0;
}

```

22. Demonstrate the use of `getpid()` and `getppid()`.

```

#include <stdio.h>
#include <unistd.h>

int main() {
    printf("PID: %d\n", getpid());
    printf("PPID: %d\n", getppid());
    return 0;
}

```


23. Write a C program that uses `waitpid()` to wait for a specific child process to terminate.

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();
    if (pid == 0) {
        printf("Child process\n");
        sleep(2);
    } else {
        waitpid(pid, NULL, 0); // Parent waits for specific child
        printf("Parent process after child exits\n");
    }
    return 0;
}
```

24. Write a C program that handles a signal (e.g., `SIGINT`).

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

void handle_signal(int sig) {
    printf("Received signal %d\n", sig);
}

int main() {
    signal(SIGINT, handle_signal);
    while (1) {
        printf("Running...\n");
        sleep(1);
    }
    return 0;
}
```

25. Write a C program that demonstrates process termination using `kill()`.

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

int main() {
    printf("PID: %d\n", getpid());
    sleep(5); // Wait 5 seconds before

    terminating kill(getpid(), SIGKILL); // Terminates the process return 0; }

---
```

This covers a wide range of topics in Shell Scripting and System Programming. Feel free to test these in your Linux environment and let me know if you have any questions or need additional explanations!

Here are more questions on **Shell Scripting** and **System Programming (Linux C Programming)** with explanations. These will help you deepen your understanding and practice more:

Shell Scripting Questions

26. Write a script that counts the number of words in a file.

```
#!/bin/bash
read -p "Enter filename: " filename
if [ -f "$filename" ]; then
    word_count=$(wc -w < "$filename")
    echo "Number of words in $filename is: $word_count"
else
    echo "File not found"
fi
```

Explanation: This script uses the `wc -w` command to count the words in a file. It takes the file as input and displays the word count.

27. Write a script to display all the files and directories in the current directory along with their permissions.

```
#!/bin/bash
ls -l
```

Explanation: This script uses the `ls -l` command, which lists the files and directories in the current directory along with their permissions, owner, group, size, and last modified time.

28. Write a script that checks whether a given number is a palindrome.

```
#!/bin/bash
read -p "Enter a number: " num
rev_num=$(echo $num | rev)
if [ "$num" -eq "$rev_num" ]; then
    echo "$num is a palindrome"
else
    echo "$num is not a palindrome"
fi
```

Explanation: This script checks if the given number is a palindrome by reversing the number using `rev` and comparing it with the original number.

29. Write a script to find the sum of all even numbers from 1 to N.

```
#!/bin/bash
read -p "Enter a number N: " N
sum=0
for ((i=2; i<=N; i+=2)); do
    sum=$((sum + i))
done
echo "Sum of even numbers from 1 to $N is $sum"
```

Explanation: This script iterates through the numbers from 1 to N, adding the even numbers to `sum`.

30. Write a script to check whether a directory exists, if not, create it.

```
#!/bin/bash
read -p "Enter directory name: " dir
if [ -d "$dir" ]; then
    echo "Directory $dir exists."
else
    mkdir "$dir"
    echo "Directory $dir created."
fi
```

Explanation: The script uses the `-d` flag to check if the directory exists. If not, it uses `mkdir` to create it.

31. Write a script that accepts a string from the user and prints it in reverse order.

```
#!/bin/bash
read -p "Enter a string: " str
rev_str=$(echo $str | rev)
echo "Reversed string: $rev_str"
```

Explanation: This script reverses the string using the `rev` command and displays the reversed string.

32. Create a script that checks if a file is empty.

```
#!/bin/bash
read -p "Enter filename: " filename
if [ ! -s "$filename" ]; then
    echo "$filename is empty"
else
    echo "$filename is not empty"
fi
```

Explanation: The `-s` flag checks if the file is not empty. If the file has no content, it returns false.

33. Write a script to rename all `.txt` files in the current directory to `.bak`.

```
#!/bin/bash
for file in *.txt; do
    mv "$file" "${file%.txt}.bak"
done
```

Explanation: This script renames all `.txt` files by removing the `.txt` extension and appending `.bak`.

34. Write a script to monitor disk usage and alert if usage exceeds a threshold.

```
#!/bin/bash
threshold=80
usage=$(df / | tail -1 | awk '{print $5}' | sed 's/%//')
if [ $usage -gt $threshold ]; then
    echo "Disk usage is above $threshold%. Usage: $usage%"
else
    echo "Disk usage is below $threshold%. Usage: $usage%"
fi
```

Explanation: This script uses `df` to get disk usage, then compares it against a threshold (80%). If the usage exceeds the threshold, it triggers an alert.

35. Create a script that takes a directory name as input and lists all files larger than a specified size.

```
#!/bin/bash
read -p "Enter directory: " dir
read -p "Enter file size in KB: " size
find "$dir" -type f -size +"$size"k -exec ls -lh {} \;
```

Explanation: This script uses `find` to search for files larger than the specified size and lists them using `ls -lh`.

System Programming (Linux C Programming) Questions

36. Write a C program to demonstrate the use of `malloc()` and `free()` to dynamically allocate memory for an array of integers.

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *arr;
    int n = 5;
```

```

arr = (int *)malloc(n * sizeof(int));
if (arr == NULL) {
    printf("Memory allocation failed\n");
    return 1;
}

// Initialize and print array
for (int i = 0; i < n; i++) {
    arr[i] = i + 1;
    printf("%d ", arr[i]);
}

free(arr);
return 0;
}

```

Explanation: This program dynamically allocates memory for an array of integers using `malloc()`, initializes the array, and then frees the memory using `free()`.

37. Write a C program that uses `fork()` to create a child process and calculates the sum of the first N numbers in both parent and child processes.

```

#include <stdio.h>
#include <unistd.h>

int main() {
    pid_t pid;
    int N = 5;
    int sum = N * (N + 1) / 2;

    pid = fork();

    if (pid == 0) {
        printf("Child process: Sum of first %d numbers is %d\n", N, sum);
    } else {
        printf("Parent process: Sum of first %d numbers is %d\n", N, sum);
    }

    return 0;
}

```

Explanation: This program uses `fork()` to create a child process. Both parent and child processes calculate and print the sum of the first N numbers.

38. Write a C program that uses `exit()` to terminate the program at a certain point.

```

#include <stdio.h>
#include <stdlib.h>

```

```
int main() {
    printf("This is the first part of the program\n");
    exit(0);
    printf("This part will not be executed\n");
}
```

Explanation: The `exit(0)` function terminates the program. The line after `exit()` will never be executed.

39. Write a C program that demonstrates the use of the `signal()` function to handle a `SIGTERM` signal.

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

void handle_sigterm(int sig) {
    printf("Received SIGTERM signal\n");
}

int main() {
    signal(SIGTERM, handle_sigterm);

    // Infinite loop to keep the program running
    while (1) {
        printf("Running... Press Ctrl+C to terminate\n");
        sleep(1);
    }

    return 0;
}
```

Explanation: This program sets up a signal handler for `SIGTERM`. When the process receives the `SIGTERM` signal, it calls the `handle_sigterm()` function.

40. Write a C program that demonstrates memory allocation and deallocation using `calloc()` and `free()`.

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *arr;
    int n = 5;

    arr = (int *)calloc(n, sizeof(int));
    if (arr == NULL) {
        printf("Memory allocation failed\n");
        return 1;
    }
}
```

```
// Initialize and print array
for (int i = 0; i < n; i++) {
    printf("%d ", arr[i]);
}

free(arr);
return 0;
}
```

Explanation: This program uses `calloc()` to allocate memory for an array, initializes the memory to 0, and prints it. Finally, it frees the memory using `free()`.

Here are more Shell Scripting and System Programming questions for your lab evaluation preparation:

Shell Scripting Questions:

1. Write a shell script to check if a given file exists. If the file exists, display its content, and if it doesn't, display an error message.
2. Write a script that takes a string input from the user and counts the number of vowels in it.
3. Create a shell script that takes a directory name as an argument and lists all the files inside that directory that have a **.txt** extension.
4. Write a script to find and delete all empty files in a given directory.
5. Create a shell script to calculate the factorial of a given number using recursion.
6. Write a script that accepts a date input in **YYYY-MM-DD** format and checks if it is a valid date.
7. Create a script that accepts a filename as an argument, reads the file line by line, and prints the total number of words in the file.
8. Write a shell script to check if a given string is a palindrome.
9. Create a script that accepts multiple arguments and outputs the number of arguments passed, their sum (if numeric), and the average (if numeric).
10. Write a shell script that asks the user to input a number and outputs whether the number is even or odd.

System Programming Questions:

1. Write a C program to print the process ID (PID) and parent process ID (PPID) of the current process.
2. Create a system program where a parent process creates two child processes. Each child process prints its PID, and the parent process waits for both to terminate.
3. Write a C program to demonstrate memory allocation using **malloc**, **calloc**, and **free**. Also, check for memory allocation errors.
4. Create a system program that creates 5 child processes. The parent process should wait for each child to terminate before it terminates itself.
5. Write a C program to use **fork()** and **exec()** to replace the current process with a new program (e.g., running **ls** command after **fork()**).
6. Write a C program to use **wait()** to ensure that the parent process waits for the termination of a child process.
7. Create a program that uses **fork()** to create a child process. The child process should terminate itself using **exit()** after printing a message.
8. Write a system program to demonstrate how memory leaks occur. Allocate memory but forget to free it.

9. Create a system program that accepts input from the user and uses `fork()` to create a child process that prints the input message.
10. Write a system program that demonstrates process creation using `fork()` where the parent process waits for the child process to finish and then exits.

Let's start by providing solutions to the questions you've requested. Given the volume of 100 questions, I'll provide solutions for a smaller batch first to ensure clarity. We'll start with the Shell Scripting and System Programming questions you requested.

System Programming Solutions

1. Print PID and PPID:

```
#include <stdio.h>

#include <unistd.h>

int main() {

    pid_t pid = getpid();

    pid_t ppid = getppid();

    printf("PID: %d, PPID: %d\n", pid, ppid);

    return 0;

}
```

2. Create child processes and parent waits:

```
#include <stdio.h>

#include <unistd.h>

#include <sys/wait.h>

int main() {

    pid_t pid1, pid2;

    pid1 = fork();

    if (pid1 == 0) {

        // First child process

        printf("Child 1 PID: %d\n", getpid());

    } else {
```

```

pid2 = fork();

if (pid2 == 0) {

    // Second child process

    printf("Child 2 PID: %d\n", getpid());

} else {

    // Parent process waits for children

    wait(NULL);

    wait(NULL);

    printf("Parent process PID: %d\n", getpid());

}

}

return 0;

}

```

3. Memory allocation using **malloc**, **calloc**, and **free**:

```

#include <stdio.h>

#include <stdlib.h>

int main() {

    int *arr;

    arr = (int *)malloc(5 * sizeof(int));

    if (arr == NULL) {

        printf("Memory allocation failed!\n");

        return 1;

    }

    for (int i = 0; i < 5; i++) {

```

```

    arr[i] = i + 1;

    printf("%d ", arr[i]);

}

free(arr);

return 0;

}

```

4. Create 5 child processes:

```

#include <stdio.h>

#include <unistd.h>

#include <sys/wait.h>

int main() {

    for (int i = 0; i < 5; i++) {

        pid_t pid = fork();

        if (pid == 0) {

            printf("Child %d PID: %d\n", i + 1, getpid());

            return 0;

        }

    }

    for (int i = 0; i < 5; i++) {

        wait(NULL);

    }

    printf("Parent process PID: %d\n", getpid());

    return 0;
}

```

```
}
```

5. Replace current process using **exec()**:

```
#include <stdio.h>
```

```
#include <unistd.h>
```

```
int main() {
```

```
    pid_t pid = fork();
```

```
    if (pid == 0) {
```

```
        // Child process will execute 'ls'
```

```
        execlp("/bin/ls", "ls", NULL);
```

```
        printf("This will not be printed.\n"); // If exec is successful
```

```
    } else {
```

```
        wait(NULL); // Parent waits for child
```

```
    }
```

```
    return 0;
```

```
}
```

6. Using **wait()** to make parent wait for child:

```
#include <stdio.h>
```

```
#include <unistd.h>
```

```
#include <sys/wait.h>
```

```
int main() {
```

```
    pid_t pid = fork();
```

```
    if (pid == 0) {
```

```
        // Child process
```

```

    printf("Child process PID: %d\n", getpid());

} else {

    // Parent process

    wait(NULL);

    printf("Parent process PID: %d\n", getpid());

}

return 0;

}

```

7. Child process using `exit()`:

```

#include <stdio.h>

#include <unistd.h>

#include <stdlib.h>

int main() {

    pid_t pid = fork();

    if (pid == 0) {

        // Child process

        printf("Child process PID: %d\n", getpid());

        exit(0); // Child exits

    }

    wait(NULL); // Parent waits

    printf("Parent process PID: %d\n", getpid());

    return 0;

}

```

8. Memory leak example:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int main() {
```

```
    int *ptr = (int *)malloc(sizeof(int) * 10);
```

```
    // Forget to free memory, causing a memory leak.
```

```
    printf("Memory allocated, but not freed.\n");
```

```
    return 0;
```

```
}
```

9. Fork and print user input in child process:

```
#include <stdio.h>
```

```
#include <unistd.h>
```

```
int main() {
```

```
    char message[100];
```

```
    printf("Enter a message: ");
```

```
    fgets(message, 100, stdin);
```

```
    pid_t pid = fork();
```

```
    if (pid == 0) {
```

```
        // Child process prints message
```

```
        printf("Child process received message: %s", message);
```

```
    }
```

```
    return 0;
```

```
}
```


10. Process creation and parent waits:

```
#include <stdio.h>

#include <unistd.h>

#include <sys/wait.h>

int main() {

    pid_t pid = fork();

    if (pid == 0) {

        // Child process

        printf("Child process PID: %d\n", getpid());

    } else {

        // Parent process waits

        wait(NULL);

        printf("Parent process PID: %d\n", getpid());

    }

    return 0;

}
```

Sure! I'll generate a comprehensive set of questions that cover the full syllabus of **Shell Scripting** and **System Programming** from the provided lab syllabus. The questions will include both conceptual and practical topics, addressing key areas such as process management, memory allocation, process creation/termination, and various shell scripting tasks.

System Programming Questions

Process Management (Chapter 6)

1. What is the difference between `fork()` and `exec()` system calls?
2. Write a C program to demonstrate the creation of a child process using `fork()`.
3. Write a C program to print the process ID (PID) and parent process ID (PPID) of a process.
4. Explain the concept of a zombie process and how it occurs.
5. What is the difference between `wait()` and `waitpid()` system calls? Provide an example for each.
6. Write a C program where a parent process creates two child processes. The parent waits for both child processes to terminate.
7. Write a C program to demonstrate the use of `getpid()` and `getppid()` functions.
8. Explain the `exit()` function and how it affects the process termination.
9. Write a program using `fork()` where the parent and child processes print different messages.
10. What is the difference between a foreground process and a background process?

Memory Allocation (Chapter 7)

11. Explain the difference between `malloc()` and `calloc()` functions.
12. Write a C program to allocate memory for an array of 10 integers using `malloc()`, assign values to the array, and then free the memory.
13. Write a C program that dynamically allocates memory for a string using `malloc()`. The program should prompt the user to input a string and then print it.
14. Write a C program that dynamically allocates memory using `calloc()` for an array of 10 integers and initializes all elements to zero.
15. What is a memory leak? How can it be prevented?
16. Write a C program that reallocates memory for an array to increase its size dynamically.
17. Explain how memory is managed in the heap and stack in a C program.

Process Creation (Chapter 24)

18. What is the purpose of the `fork()` system call in process creation?
19. Write a C program that creates a child process and uses `execl()` to replace the child process image with another program.
20. Explain the purpose and use of `execv()` and `execl()` system calls.
21. Write a C program that creates a child process and replaces it with a new program using `execvp()`.
22. Explain the difference between `exec()` family functions (`execl()`, `execv()`, `execp()`, `execvp()`).
23. What is a process control block (PCB) and what information does it store?
24. Explain the concept of process scheduling and how the operating system schedules processes.

Process Termination (Chapter 25)

25. Write a C program to demonstrate process termination using `exit()`.
26. What happens when a process terminates? Discuss the role of the exit status.
27. Write a program that demonstrates the use of `wait()` system call to make the parent wait for the termination of a child process.
28. What is a daemon process? Explain how it differs from regular processes.
29. Explain how a process is terminated when it encounters a `SIGKILL` signal.

Monitoring Child Processes (Chapter 26)

30. Write a C program that uses `waitpid()` to monitor a child process's termination.
31. What is the role of the `SIGCHLD` signal in process monitoring?
32. Write a C program that demonstrates the use of `signal()` to handle `SIGCHLD` signals.
33. Explain how `wait()` and `waitpid()` can be used to monitor multiple child processes.

General System Programming

34. Explain the difference between system calls and library functions with examples.
35. Write a C program to create a pipe between two processes, where one process sends data and the other reads it.
36. What is the role of the `select()` system call in multiplexing input/output?
37. Write a C program that uses the `select()` function to monitor multiple file descriptors for input.
38. What is a deadlock in process management? How can it be prevented?

39. Explain how a shared memory segment is created using `shmget()` and accessed using `shmat()`.
 40. Write a C program that demonstrates the use of `mmap()` for memory-mapped file I/O.
-

Comprehensive Practical Problems

1. Write a shell script that checks if a directory exists and creates it if it doesn't. Then, create multiple text files in that directory.
 2. Write a C program to create 5 child processes, where each child process prints its PID and its parent's PID. The parent waits for all children to terminate.
 3. Write a C program that replaces the current process with the `ls` command using `execlp()`.
 4. Write a shell script to automate the backup of user files to a designated backup directory.
 5. Write a C program that creates a child process using `fork()` and demonstrates inter-process communication using pipes.
-

These questions cover a broad spectrum of topics, from basic concepts to advanced system programming and shell scripting tasks. Let me know if you'd like more specific examples, or if you'd prefer further assistance with solving any particular questions.

System Programming Solutions

Process Management (Chapter 6)

1. Difference Between `fork()` and `exec()`

- `fork()` creates a new child process by duplicating the parent process.
- `exec()` replaces the current process image with a new program.

Create Child Process Using `fork()`

```
#include <stdio.h>
#include <unistd.h>
int main() {
    pid_t pid = fork();
    if (pid == 0) {
        printf("Child Process: PID = %d\n", getpid());
    } else {
        printf("Parent Process: PID = %d, Child PID = %d\n", getpid(), pid);
    }
    return 0;
}
```

2.

Print PID and PPID

```
#include <stdio.h>
#include <unistd.h>
int main() {
    printf("PID = %d, PPID = %d\n", getpid(), getppid());
    return 0;
}
```

3.

4. **Zombie Process Explanation** A zombie process occurs when a child process terminates, but its parent process has not yet called `wait()` to collect its exit status.

5. `wait()` vs `waitpid()`

- `wait()` waits for any child process to terminate.
- `waitpid()` can wait for a specific child process to terminate and provides more control.

Parent Creates Two Child Processes

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
```

```

int main() {
    pid_t pid1 = fork();
    if (pid1 == 0) {
        printf("Child 1: PID = %d\n", getpid());
        return 0;
    }
    pid_t pid2 = fork();
    if (pid2 == 0) {
        printf("Child 2: PID = %d\n", getpid());
        return 0;
    }
    wait(NULL);
    wait(NULL);
    printf("Parent: PID = %d\n", getpid());
    return 0;
}

```

6.

getpid() and getppid()

```

#include <stdio.h>
#include <unistd.h>
int main() {
    printf("PID = %d, PPID = %d\n", getpid(), getppid());
    return 0;
}

```

7.

Process Termination Using exit()

```

#include <stdio.h>
#include <stdlib.h>
int main() {
    printf("Process is terminating...\n");
    exit(0);
}

```

8.

Parent and Child Print Different Messages

```

#include <stdio.h>
#include <unistd.h>
int main() {
    pid_t pid = fork();
    if (pid == 0) {
        printf("Child Process\n");
    } else {
        printf("Parent Process\n");
    }
    return 0;
}

```

9.

Memory Allocation (Chapter 7)**11. malloc() vs calloc()**

- **malloc()** allocates uninitialized memory, while **calloc()** allocates zero-initialized memory.

12. Dynamic Memory Allocation

```
#include <stdio.h>
#include <stdlib.h>
int main() {
    int *arr = (int *)malloc(5 * sizeof(int));
    if (arr == NULL) {
        printf("Memory allocation failed.\n");
        return 1;
    }
    for (int i = 0; i < 5; i++) {
        arr[i] = i + 1;
    }
    for (int i = 0; i < 5; i++) {
        printf("%d ", arr[i]);
    }
    free(arr);
    return 0;
}
```

13. Use of free()

- **free()** is used to deallocate memory that was previously allocated by **malloc()**, **calloc()**, or **realloc()**.

Process Creation and Termination (Chapters 24-26)**14. Create Child Process and Replace Using exec()**

```
#include <stdio.h>
#include <unistd.h>
int main() {
    pid_t pid = fork();
    if (pid == 0) {
        execlp("/bin/ls", "ls", NULL);
    } else {
        wait(NULL);
    }
    return 0;
}
```

Let me know if you need further details or more practice questions!